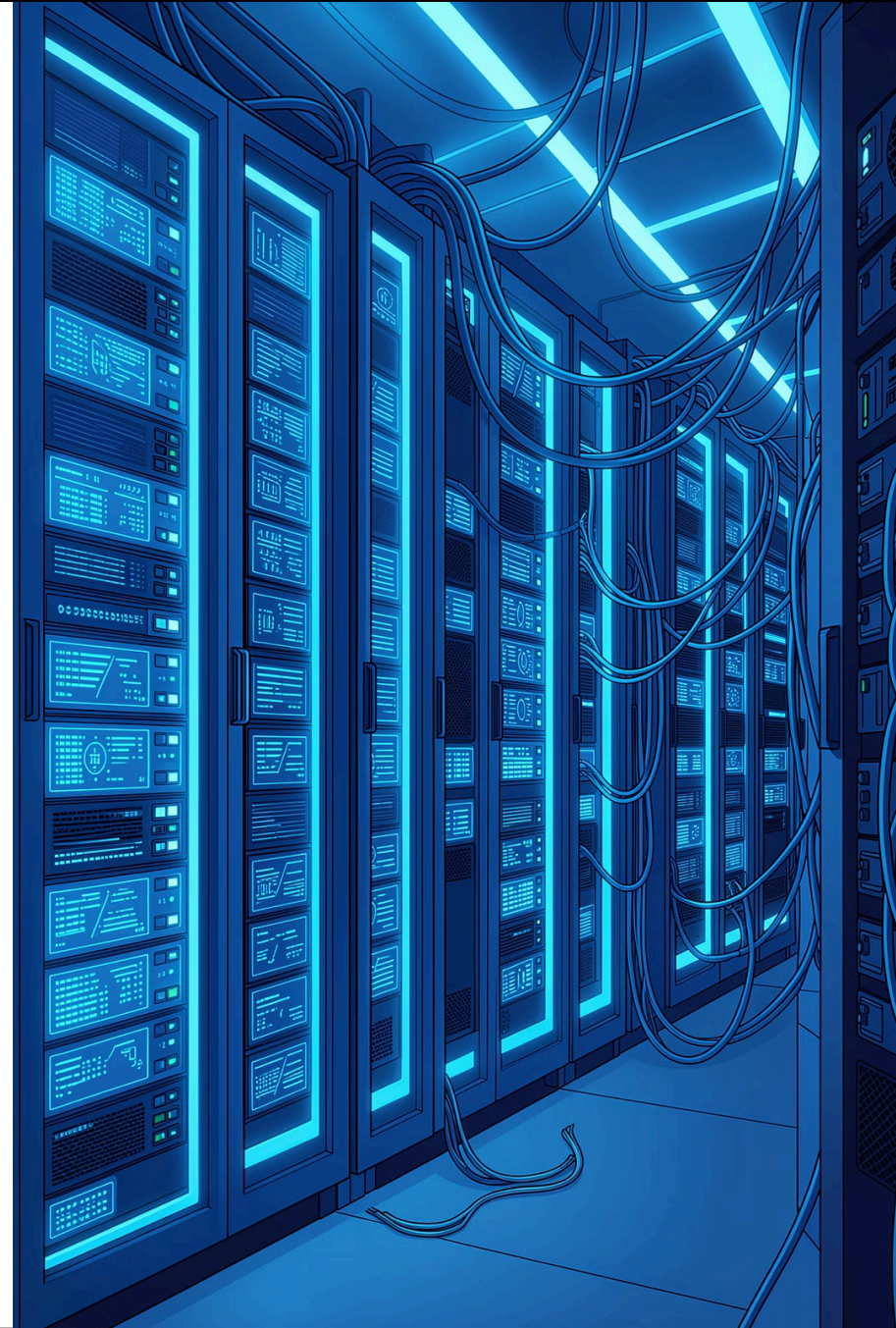


Semana 6 — PHP, POO y MVC en Acción

Bienvenidos a la clase más importante del taller hasta ahora. Hoy vamos a construir desde cero los fundamentos de PHP, entender la Programación Orientada a Objetos, y dominar el patrón MVC — todo conectado a un caso real: un **Dashboard de Monitoreo de Activos** que corre en producción en datacenters reales.

TALLER DE APLICACIONES PARA INTERNET

SEMANA 6 · PROF. ROBERTO THIERS





¿Por Qué PHP? Contexto e Historia

PHP nació en 1994 como un conjunto de scripts personales de Rasmus Lerdorf. Hoy, más de **30 años después**, impulsa el **77% de los sitios web del mundo** con backend conocido — incluyendo Facebook en sus inicios, Wikipedia, y WordPress.

¿Por qué sigue siendo relevante? Porque es fácil de aprender, corre en cualquier servidor, y tiene un ecosistema maduro con frameworks como **Laravel** que lo mantienen moderno y competitivo.



77%

de sitios web usan PHP



Laravel

Framework más popular del mundo web



Open Source

Gratis, extensible y con enorme comunidad

Arquitectura: Cliente → Servidor → HTTP

Antes de escribir una sola línea de PHP, hay que entender **cómo funciona el web**. Todo empieza con una solicitud y termina con una respuesta. PHP vive exactamente en el medio — en el servidor.



El browser nunca ve tu código PHP. Solo recibe el HTML que PHP generó. Esta separación entre lógica y presentación es el corazón de todo lo que aprenderemos hoy.

Características de PHP: ¿Qué lo hace especial?



Server-Side

Corre en el servidor, no en el navegador. El cliente nunca ve tu lógica ni tus credenciales de base de datos.



Embebido en HTML

Puedes mezclar PHP con HTML directamente. Comienza con `<?php` y termina con `?>`.



Multiplataforma

Corre en Linux, Windows y macOS. Compatible con Apache, Nginx y más servidores web.



Integración con BD

Conexión nativa con MySQL, PostgreSQL, SQLite y más. Perfecto para apps con datos dinámicos.



Tipado Dinámico

No necesitas declarar el tipo de una variable. PHP lo infiere automáticamente según el valor asignado.



Comunidad Global

Millones de desarrolladores, documentación exhaustiva en php.net y miles de librerías disponibles.

XAMPP: Tu Primer Ambiente de Desarrollo

¿Qué es XAMPP?

XAMPP es un paquete gratuito que instala todo lo que necesitas para desarrollar en PHP localmente: **Apache** (servidor web), **MySQL/MariaDB** (base de datos), y **PHP** — todo en un solo instalador.

¿Por qué usarlo?

Simula un servidor real en tu computador. Puedes desarrollar y probar sin conexión a internet ni hosting pagado.

Tu stack local completo

● Apache

Servidor HTTP que procesa las peticiones web

● MySQL

Base de datos relacional para almacenar datos

● PHP

Motor que ejecuta tu lógica de negocio

♥ VS Code

Editor de código con extensiones para PHP



Sintaxis Básica: Variables y Echo

En PHP, todas las variables comienzan con \$. No necesitas declarar el tipo — PHP lo determina por el valor. Para mostrar contenido en pantalla usamos echo o print.

Variables y tipos de datos

```
<?php
// String
$nombre = "Roberto";

// Integer
$edad = 42;

// Float
$precio = 99.90;

// Boolean
$activo = true;

// Array
$cursos = ["PHP", "Laravel", "JS"];

echo "Hola, " . $nombre;
echo "<br>";
echo "Tienes " . $edad . " años";
?>
```

Operadores comunes

Tipo	Operador	Ejemplo
Suma	+	\$a + \$b
Concatenar	.	"Hola"."Mundo"
Igual	==	\$a == 5
Idéntico	===	\$a === "5"
Y lógico	&&	\$a && \$b
O lógico		\$a \$b

Usa === para comparar también el tipo de dato. Es más seguro que ==.

Control de Flujo: If/Else y Loops

El control de flujo determina **qué hace tu programa según las condiciones**. En PHP los condicionales y loops son muy similares a JavaScript o Java — si ya los conoces, esto te va a resultar familiar.

Condicionales

Loops

```
<?php
// foreach con array
$servidores = ["SRV01","SRV02","SRV03"];
foreach ($servidores as $srv) {
    echo $srv . " — Online <br>";
}

// for clásico
for ($i = 1; $i <= 5; $i++) {
    echo "Iteración " . $i . "<br>";
}
?>
```

📌 ¿Lo ven? Ya estamos usando variables de servidores. Todo lo que aprendemos hoy aparece en el Dashboard de Monitoreo al final. 🚀

Funciones en PHP

Las funciones son bloques de código reutilizable. En PHP se definen con la palabra clave `function`. Pueden recibir parámetros, retornar valores, y tener valores por defecto — esenciales para escribir código limpio y organizado.

```
<?php
// Función básica con parámetros
function saludar($nombre, $rol = "Estudiante") {
    return "Hola, " . $nombre . " (" . $rol . ")";
}

echo saludar("Ana");
// → Hola, Ana (Estudiante)

echo saludar("Roberto", "Profesor");
// → Hola, Roberto (Profesor)

// Función que valida temperatura de servidor
function checkTemp($temp) {
    if ($temp >= 90) return "CRÍTICO";
    if ($temp >= 75) return "ALERTA";
    return "OK";
}

echo checkTemp(92); // → CRÍTICO
echo checkTemp(70); // → OK
?>
```

Buenas prácticas

→ Una función, una responsabilidad

Cada función debe hacer una sola cosa bien hecha.

→ Nombres descriptivos

Usa verbos: `getUser()`, `validateEmail()`, `saveData()`.

→ Siempre usa return

Las funciones que calculan algo deben retornar el resultado, no solo mostrarlo.

Caso de Éxito: Validación de Formularios

Los formularios son el **punto de entrada de datos del usuario** en cualquier aplicación web. PHP recibe esos datos via `$_POST` o `$_GET` y los valida antes de procesarlos. Nunca confíes en los datos del usuario sin validar.

Reglas de validación

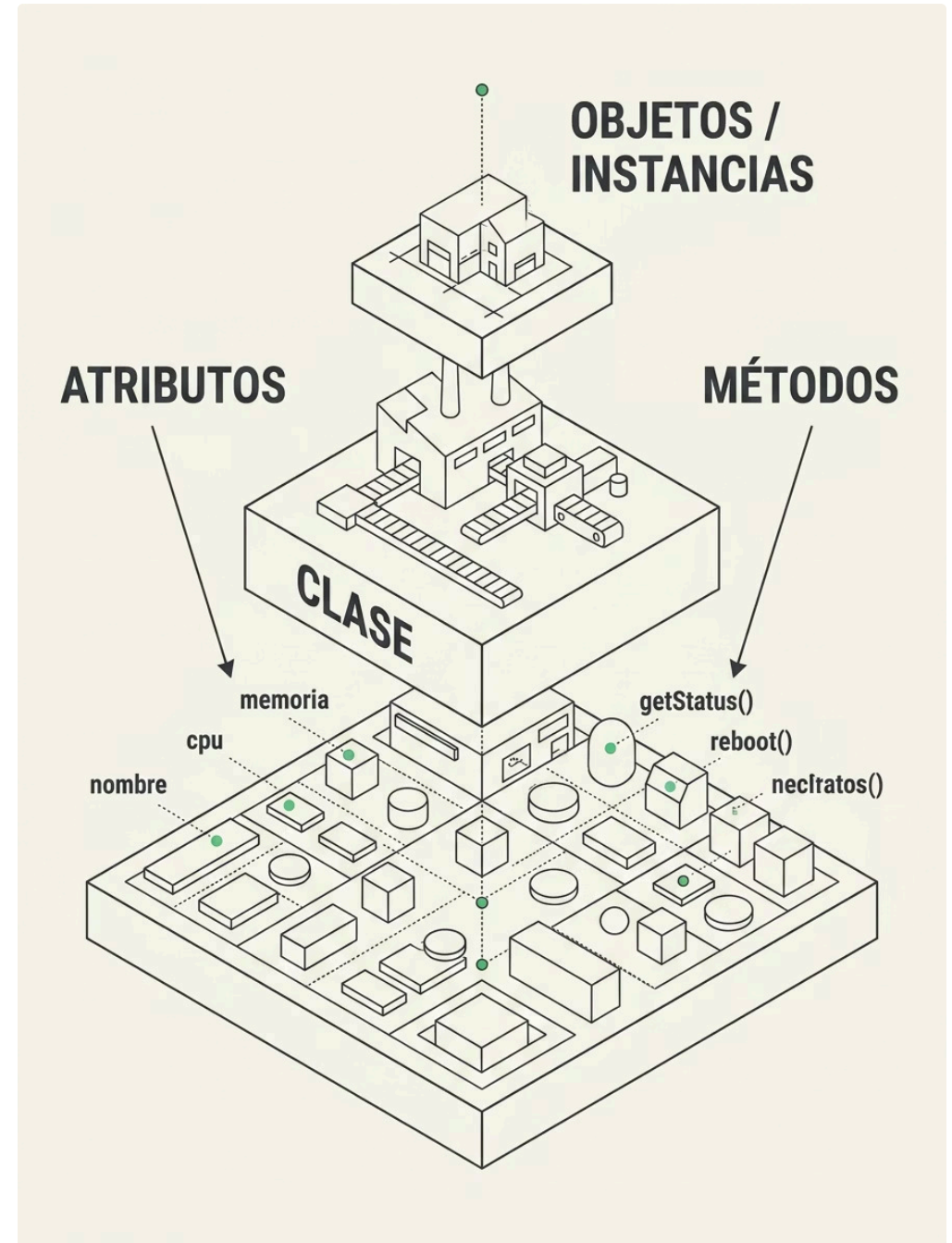
- **Nunca confíes** en datos del cliente — siempre valida en el servidor
- Usa `htmlspecialchars()` para prevenir XSS
- Usa `filter_var()` para emails, URLs, números
- Usa `trim()` para limpiar espacios innecesarios
- Verifica con `empty()` que los campos obligatorios estén llenos

Programación Orientada a Objetos: ¿Qué es una Clase?

Hasta ahora programamos de forma **procedimental**: instrucciones en secuencia. La **Programación Orientada a Objetos (POO)** cambia el enfoque: modelamos el mundo real usando **clases y objetos**.

Una **clase** es como un plano o molde. Un **objeto** es una instancia concreta de ese plano. La clase define qué atributos tiene y qué puede hacer. Los objetos son las instancias reales que creamos.

❗ Piensa en una clase **Servidor** como el plano. Cada servidor físico en el datacenter (SRV01, SRV02...) es un **objeto** diferente, creado desde ese mismo plano.



Objetos PHP: Clase Car en Vivo

Veamos un ejemplo concreto y fácil de entender antes de pasar a casos empresariales. La clase `Car` tiene atributos que describen el auto y métodos que definen lo que puede hacer.

```
<?php
class Car {
    // Atributos (propiedades)
    public $marca;
    public $modelo;
    public $año;
    private $velocidad = 0;

    // Constructor: se ejecuta al crear el objeto
    public function __construct($marca, $modelo, $año) {
        $this->marca = $marca;
        $this->modelo = $modelo;
        $this->año = $año;
    }

    // Métodos (comportamiento)
    public function acelerar($cantidad) {
        $this->velocidad += $cantidad;
        echo "Acelerando a " . $this->velocidad . " km/h";
    }

    public function getInfo() {
        return $this->marca . " " . $this->modelo . " (" . $this->año . ")";
    }
}

// Crear objetos (instanciar)
$auto1 = new Car("Toyota", "Corolla", 2023);
$auto2 = new Car("Ford", "Mustang", 2024);

echo $auto1->getInfo(); // Toyota Corolla (2023)
$auto1->acelerar(60); // Acelerando a 60 km/h
?>
```

Conceptos clave

`$this`

Referencia al objeto actual. Accede a sus propios atributos y métodos.

`new`

Crema una nueva instancia (objeto) a partir de la clase.

`->`

Operador para acceder a propiedades y métodos del objeto.

Constructor y Patrón GET/SET

El **constructor** inicializa el objeto cuando se crea. Los **getters y setters** son métodos que controlan el acceso a atributos privados — una práctica fundamental para proteger la integridad de los datos en tu aplicación.

```
<?php
class Servidor {
    private $nombre;
    private $ip;
    private $cpuUsage;

    public function __construct($nombre, $ip) {
        $this->nombre = $nombre;
        $this->ip = $ip;
        $this->cpuUsage = 0;
    }

    // GETTER: leer propiedad privada
    public function getNombre() { return $this->nombre; }
    public function getIp() { return $this->ip; }
    public function getCpuUsage() { return $this->cpuUsage; }

    // SETTER: escribir con validación
    public function setCpuUsage($value) {
        if ($value >= 0 && $value <= 100) {
            $this->cpuUsage = $value;
        } else {
            throw new Exception("CPU usage inválido: " . $value);
        }
    }
}

$srv = new Servidor("SRV-01", "192.168.1.10");
$srv->setCpuUsage(75);
echo $srv->getNombre() . " CPU: " . $srv->getCpuUsage() . "%";
// → SRV-01 CPU: 75%
?>
```

¿Por qué private?

Encapsulamiento

Protege los datos internos del objeto de modificaciones externas no controladas.

Validación

El setter puede validar antes de asignar. Nadie puede poner CPU = 200%.

Mantenibilidad

Si cambias la lógica interna, el código externo no se rompe.

Herencia: La Realidad en Clases PHP

La **herencia** permite crear clases nuevas basadas en clases existentes. La clase hija hereda todos los atributos y métodos del padre, y puede agregar los suyos propios o sobrescribir el comportamiento. En nuestro caso de uso, ServidorWeb y ServidorBD heredan de Servidor.

```
<?php
// Clase padre (base)
class Servidor {
    protected $nombre;
    protected $ip;
    public function __construct($nombre, $ip) {
        $this->nombre = $nombre;
        $this->ip = $ip;
    }
    public function getStatus() { return $this->nombre . " — Activo"; }
}

// Clase hija: hereda de Servidor
class ServidorWeb extends Servidor {
    private $dominio;
    public function __construct($nombre, $ip, $dominio) {
        parent::__construct($nombre, $ip); // llama constructor padre
        $this->dominio = $dominio;
    }
    public function getStatus() { // sobrescribe el método padre
        return parent::getStatus() . " | Web: " . $this->dominio;
    }
}

$srvWeb = new ServidorWeb("WEB-01", "10.0.0.1", "datacenter.cl");
echo $srvWeb->getStatus();
// → WEB-01 — Activo | Web: datacenter.cl
?>
```

✔ Con `extends` heredas todo. Con `parent::` accedes al código del padre. Con `protected`, las propiedades son accesibles desde las clases hijas.

Caso POO: Estructura de Datos del Negocio

En aplicaciones reales, cada entidad del negocio se modela como una clase. Veamos tres clases clave que aparecerán en formularios y bases de datos reales.



Clase Persona

Modela a los usuarios del sistema.

- `$rut`, `$nombre`, `$email`
- Método `validarRut()`
- Método `toArray()` para formularios



Clase Localidad

Modela regiones y comunas.

- `$region`, `$comuna`, `$codigoPostal`
- Método `getFullAddress()`
- Relación: Persona tiene una Localidad



Clase Producto

Modela ítems del catálogo.

- `$sku`, `$nombre`, `$precio`, `$stock`
- Método `aplicarDescuento()`
- Método `hayStock()`

Estas clases son la base del **Modelo** en MVC. Cuando pasemos a la siguiente sección, verás cómo estas mismas clases se conectan con la base de datos y con la vista del usuario.

NUEVO TEMA

Patrón MVC: Separación de Responsabilidades

El patrón **Modelo-Vista-Controlador** es la arquitectura más usada en aplicaciones web profesionales. Su propósito es simple pero poderoso: **separar lo que hace cada parte del sistema** para que sea más fácil de mantener, escalar y trabajar en equipo.



Modelo

Gestiona los datos y la lógica de negocio. Se conecta a la base de datos. No sabe nada de HTML ni de interfaz.



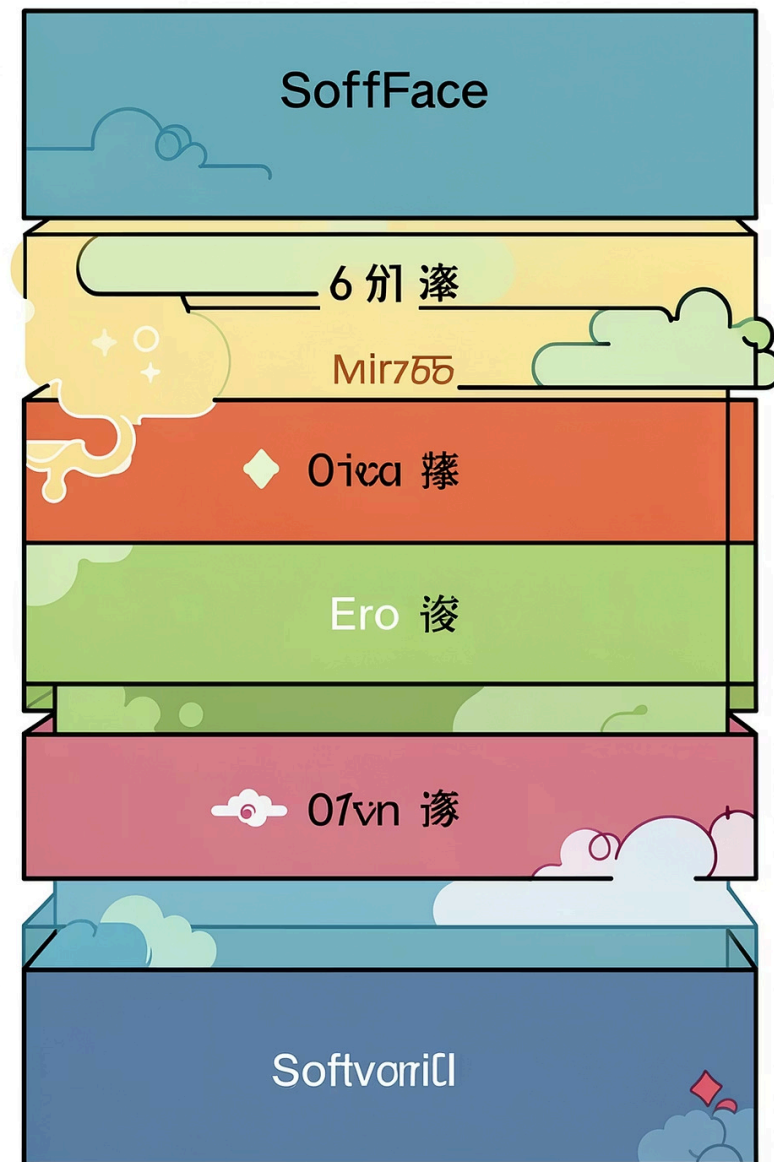
Controlador

El intermediario. Recibe peticiones del usuario, consulta al Modelo, y decide qué Vista mostrar con qué datos.

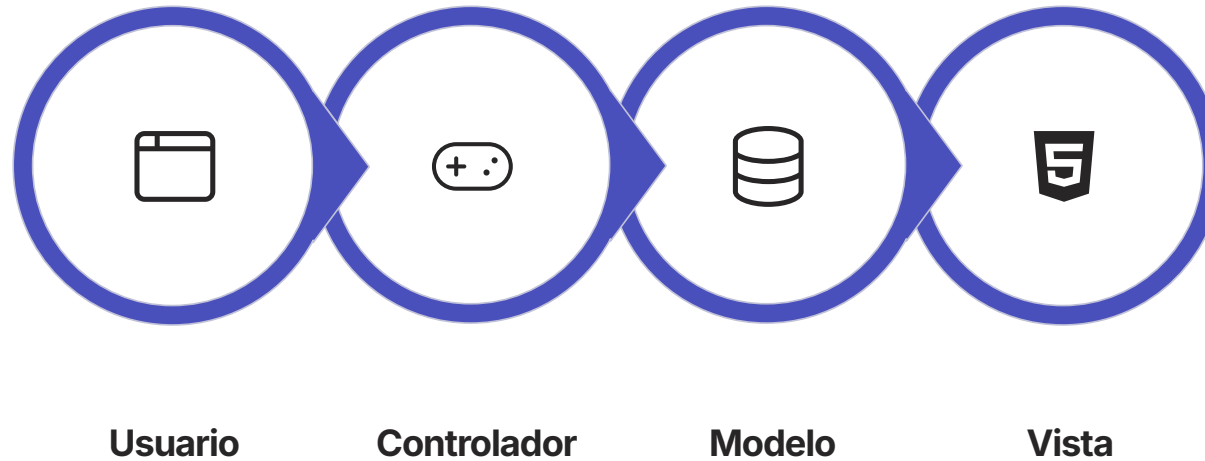


Vista

Presenta los datos al usuario. Solo HTML y presentación. No hace cálculos ni consultas a la BD directamente.



Componentes MVC: El Flujo Completo



Modelo en PHP

```
class ServidorModel {  
    public function getAll() {  
        // Consulta a BD  
        return $servidores;  
    }  
}
```

Controlador

```
class MonitorController {  
    public function index() {  
        $m = new ServidorModel();  
        $data = $m->getAll();  
        include "views/dashboard.php";  
    }  
}
```

Vista

```
<!-- dashboard.php -->  
<?php foreach($data as $s): ?>  
    <div>  
        <?= $s->nombre ?>  
    </div>  
<?php endforeach ?>
```

Implementación MVC: AutoController Ejemplo

Veamos un controlador completo en acción. El `AutoController` gestiona el ciclo de vida de los autos: listar, crear, editar y eliminar (el famoso **CRUD**). Cada método del controlador corresponde a una URL diferente.

```
<?php
class AutoController {

    private $model;

    public function __construct() {
        $this->model = new AutoModel();
    }

    // GET /autos → lista todos
    public function index() {
        $autos = $this->model->getAll();
        include "views/autos/index.php";
    }

    // GET /autos/create → formulario nuevo
    public function create() {
        include "views/autos/create.php";
    }

    // POST /autos/store → guarda nuevo
    public function store() {
        $data = [
            'marca' => $_POST['marca'],
            'modelo' => $_POST['modelo'],
            'año' => $_POST['año']
        ];
        $this->model->save($data);
        header("Location: /autos");
    }

    // GET /autos/delete/{id}
    public function delete($id) {
        $this->model->delete($id);
        header("Location: /autos");
    }
}
?>
```

El patrón CRUD

Acción	Método HTTP
Listar	GET
Crear form	GET
Guardar	POST
Editar form	GET
Actualizar	PUT/POST
Eliminar	DELETE/POST

 Este mismo patrón se repite para *cualquier entidad*: Servidor, Usuario, Producto, Alerta. Una vez que lo dominas, escalas ilimitado.

Dashboard MVC: Arquitectura de Carpetas

Una buena estructura de carpetas es la diferencia entre un proyecto mantenible y un caos. En el proyecto del Dashboard de Monitoreo, organizamos así:



Modelos del Dashboard

Clase Servidor

Atributos: \$nombre, \$ip, \$cpu, \$memoria, \$disco, \$temperatura, \$status

Clase Alerta

Atributos: \$tipo, \$severidad, \$mensaje, \$timestamp, \$servidorId

Clase Métrica

Atributos: \$servidorId, \$tipo, \$valor, \$unidad, \$timestamp

WebSockets en Tiempo Real: El Dashboard Vivo

El gran diferenciador del Dashboard es que los datos se actualizan **en tiempo real sin recargar la página**. Esto es posible gracias a **WebSockets**: una conexión persistente entre el browser y el servidor que permite comunicación bidireccional.

JavaScript (Vista) — WebSocket client

¿Por qué WebSockets y no AJAX?

Sin polling

No necesitas consultar el servidor cada N segundos. El servidor te avisa cuando hay cambios.

Bidireccional

El servidor puede enviar datos al cliente en cualquier momento, sin esperar petición.

Baja latencia

Latencia de milisegundos. Crítico para monitoreo de alertas en datacenters reales.

Conclusión: Laravel y el Futuro de PHP

Hoy recorrieron el camino completo: desde echo "Hola Mundo" hasta un Dashboard de monitoreo empresarial con WebSockets y patrón MVC. Eso es exactamente lo que se usa en producción en bancos, datacenters y plataformas de streaming.



PHP Básico

Variables, funciones, formularios — la base de todo.



Patrón MVC

Separación de responsabilidades — arquitectura profesional.



Orientación a Objetos

Clases, herencia, encapsulamiento — código organizado y escalable.



Laravel — Próximo Paso

Todo lo aprendido hoy es exactamente la base de Laravel. MVC, ORM Eloquent, Blade templates. ¡Están listos!

Lo que aprendieron HOY es lo que se usa en empresas REALES. No es teórico — es PRÁCTICO. ¡Nos vemos la Semana 7! 🚀 — Prof. Roberto Thiers